A meta-heuristic implementation for the Maximum Boolean Satisfiability Problem

Ian Gabriel, Carlos Eduardo

[1] Federal University of Rio Grande do Norte, Natal, Brazil

Abstract. *This report presents a meta-heuristic implementation of an algorithm for the maximum satisfiability problem (MAX-SAT), together with experimental results and comparison with previous works. We also discuss the current meta-heuristic available for this problem. It is worth mentioning that this is a graded assignment for the discipline of Algorithms Analysis and Projects of the Federal University of Rio Grande do Norte.*

1. Introduction

The maximum boolean satisfiability problem (MAX-SAT) involves satisfying the most amount of clauses of a logical expression in the Conjunctive Normal Form (CNF). It is a version of the boolean satisfiability problem (SAT), where you have to determine if a given boolean expression is satisfiable. Despite the straightforward definition of MAX-SAT, it has several applications across several fields, for example, it can be used for AI optimization [CoreO Research Group 2016], bioinformatics [Strickland et al. 2005], eletronics [Sandholm 1999], sports scheduling [Miyashiro and Matsui 2006] and routing [Xu et al. 2002]. These diverse applications demonstrate the problem's impact across both theoretical and practical worlds.

MAX-SAT can be viewed as an optimization version of the SAT, which is a decision problem proved to be NP-complete [Cook 1971]. Therefore, MAX-SAT is considered a NP-hard problem, since its solution implies the solution of SAT. It follows that if such solution where achieved, then it would prove that P=NP, leading to a great advance in the computer science field.

The report is structured as follows: Section 2 defines the MAX-SAT problem. In Section 3, we present the state-of-the-art in meta-heuristics for MAX-SAT solvers. Section 4 explains about Genetic Algorithm and Taboo Search, the meta-heuristics we developed for this problem. Section 5 contains the results of the empirical analysis and the comparison with an exact algorithm and an heuristic developed in previous works. The results are dicussed in Section 6 and Section 7 concludes this report.

2. Background

2.1. Conjuctive Normal Form (CNF)

A propositional logic formula is said to be in the Conjuctive Normal Form if it has the form:

$$P_1 \wedge P_2 \wedge \cdots \wedge P_n$$

where P_i is a disjunction of positive or negative literals, called a clause. Formally, P_i is written as:

$$Q_1 \vee Q_2 \vee \cdots \vee Q_n$$

and Q_i is x or $\neg x$ for a variable x.

An example of CNF formula is: $(x \vee y \vee \neg z) \wedge (x \vee \neg y) \wedge (\neg w \vee x)$

We say a clause C is satisfied by a set of assignments X of C's variables, if the resulting logical value of the expression after the assignments is 1, i.e true. For example, given $C = (x \vee y \vee \neg z)$ and $X = \{(x, 1)\}$, then $C = 1$, thus X satisfies C.

It is worth mentining that a common notation to express these formulas is the set notation. In this notation, the formula is represented as a set of clauses. Each clause is represented as a set of literals. For example, the formula $\Delta = (x \vee y \vee \neg z) \wedge (x \vee \neg y) \wedge (\neg w \vee x)$ can be written as follows:

$$\Delta = \{\{x, y, \neg z\}, \{x, \neg y\}, \{\neg w, x\}\}$$

2.2. MAX-SAT problem

The maximum satisfiability problem is defined as follows: Given a CNF formula, return an assignment of the variables as to satisfy the most amount of clauses in the formula.

3. State-of-the-Art

In our research, we found that the state-of-the-art of MAX-SAT solvers using meta-heuristics, was mostly made up of Stochastic Local Search (SNS) [CHU et al. 2019, Chu et al. 2023, Whitley et al. 2013, Alkasem and Menai 2021], with a few implementations of Variable Neighborhood Search [Bouhmala 2016, Bouhmala 2014], Dynamic Local Search (DLS) [Cai and Lei 2020], Genetic, [Bhattacharjee and Chauhan 2017] and Memetic algorithms [Bouhmala 2012], also with Simulated Annealing [Bouhmala 2019] and Particle-Swarm [Djenouri et al. 2019] approaches. In light of this, we have limited our scope to detailing 5 algorithms, which we will present in following subsections, except for the Genetic Algorithm, which will be discussed in Section 4.

3.1. CCBMS

CCBMS is a Stochastic Local Search Algorithm that performed well on solving industrial instances [CHU et al. 2019], unlike previous state-of-the-art SLS algorithms for MAX-SAT. It is an improvement over CCLS [Luo et al. 2015] achieved by using the BMS [Cai 2015] heuristic for choosing variables. Therefore, we will first explain CCLS and BMS, before explaining CCBMS.

The CCLS algorithm is also a SLS which first generates a random solution s^* for the given clause set, then it iterates, choosing a variable to flip in s^* each time, until it exceeds a time limit or s^* satisfies all clauses. During the variable selection part, it either does a random walk — i.e chooses a random variable from an unsatisfied clause — with probability p (where p is a parameter), or calls CCM, a function that returns the variable with a highest score, with probability $1 - p$. The score of a variable is defined by the number of satisfied clauses minus the number of unsatisfied clauses caused by its flip. Then, the chosen variable is only flipped if it leads to a greater number of satisfied clauses, else it skips to the next iteration. At the end, it returns s^*.

The BMS heuristic consists of choosing the best of k variables from a certain set S, where "best" is defined through a function f received as input, such that it compares

variables. This algorithm loops for k iterations, choosing a random variable from S and storing the best variable s until that point, then it returns s.

The CCBMS algorithm differs from CCLS in the variable selection part, more specifically, when where the variable is chosen through CCM (the case with probability of $1 - p$). It now relies on a parameter k which is initialized before the main loop with value k_min. If k is greater to another parameter k_max, then it will choose CCM, else it will increment k by 10 and if $k \leq k\_max$, choose BMS, otherwise it will choose CCM. The arguments used in BMS are k, MPvars (a set of variables that if flipped would increase the number of satisfied clauses) and a function that chooses variables with the highest score.

This combination of CCLS and BMS was intended to improve the algorithm's run time in industrious instances, where the number of clauses and variables far surpassed the ones in crafted instances. It achieved this by using BMS, which is fast, in the early stages of the search, guaranteeing a fast convergence, and then switching to CCM in the late stage since it was better at choosing variables than BMS.

3.2. SATLike

SATLike is a Dynamic Local Search algorithm (DLS) that performed well in recent MAX-SAT competitions [Cai and Lei 2020]. It is currently on its third iteration (SATLike 3.0), where it utilizes Unit propagation (UP) — assigning variables in unit clauses until there is none or a conflict — in a greedy manner for better performance.

The algorithm was made for the Weighted Partial MAX-SAT problem, where the clauses have weights and are either Hard or Soft. The solution must satisfy all the Hard clauses, but not necessarily the Soft ones, while also maximizing the weights. It is worth mentioning that this problem is a superset of MAX-SAT, since one could simply set the all the weights to 1 and make all the clauses Soft to obtain the original problem, therefore SATLike can also be applied to MAX-SAT.

Essentially, SATLike iterates until there are no unassigned variables left. During each iteration, it prioritizes assigning variables that satisfy Hard unit clauses, before Soft ones. In each case, it checks if the selected clause C has a contradictory counter part — a clause that if satisfied, unsatisfies C and vice-versa. If such clause exists, it chooses a variable related to both clauses and assigns it, such that it would cause the most amount of satisfied clauses, else it performs unit propagation.

3.3. SA-KL/rw-k

The SA-KL/rw-k algorithm [Bouhmala 2019] is a Simulated Annealing (SA) approach for MAX-SAT. It utilizes concepts from the Kernighan–Lin (KL) algorithm for partitioning graphs minimizing the cost of the cut edges.

It first generates a random assignment of variables $A_{[current]}$ and then starts iterating — altering this assignment — until a stop condition is reached. In each iteration it first does the KL step, which consists of sequentially saving the $gain_{v_i}$ for each variable v_i, ordered from best to worst. The quality of a variable v_i is determined by a maximal decrease or minimal increase in the number of unsatisfied clauses in case v_i is flipped, and the $gain_{v_i}$ consists of the number of satisfied clauses minus the number of unsatisfied ones caused by flipping v_i. After the KL step, a parameter k is chosen such that it

minimizes the following sum: $TotalGain = \sum_{j=1}^{k} gain_{v_j}$. Then if $TotalGain \leq 0$, the temperature is reduced and all flipped variables are unflipped, else it will flip $v1$ — i.e the best variable chosen at the KL step — in case $random(0,1) \leq exp^{gain_{v_1}}/T$, where T is the temperature and $random(0,1)$ returns a random number from the interval $(0,1)$. Otherwise it will perform a random kick on $A_{[current]}$.

3.4. MLBSO

MLBSO is Bee-Swarm optimization algorithm (BSO) fused with the Multilevel Paradigm (ML) — normally utilized in graph partitioning problems. We will first explain BSO and ML, then MLBSO.

The BSO algorithm takes inspiration from the collective foraging behavior of bees, more specifically, the fact that each bee explores it's own local search space and shares the information with the rest of the colony. By repeating this procedure, the desired information (in this case a MAX-SAT solution) is optimized, making it a promising algorithmic strategy.

The ML Paradigm consists of three phases: Coarsening Phase, Initialization Phase and Extension and Refinement Phase. The first consists of iteratively coarsening the solution space — that is, making it smaller — until further coarsening makes a solution invalid. The second phase consists of generating a random solution from the most coarse space. In the third and final phase, it repeatedly extends a given solution (initially the solution from the previous phase), bringing it to the original uncoarsened space and then refines it with some method, increasing it's quality.

MLBSO is a essentially a version of ML using BSO in the refinement phase. For the Coarsening Phase, it constructs clusters of variables, treating them as a single variable — an assignment to the cluster will result in the same assignment for all it's variables. Then, after generating an initial solution by randomly assigning values to the clusters, it goes into the third phase. In it, BSO is utilized by assigning k bees each one with a distinct neighboring solution from the current solution, generated by flipping a $Flip$ amount of variables.

4. Our Meta-heuristics

For this work, there were two attempts for solving the SAT problem applying meta-heuristics. The first algorithm uses Taboo Search to avoid repeating solutions and thus diversifying the number of solutions tried. The second attempt is a genetic algorithm that finds and improves good solutions trying to find a perfect variable assignment.

4.1. Taboo search

Our Taboo Search algorithm is based on a simple outline of a Taboo Search presented at [Boughaci and Drias 2005]. This was our initial attempt at a MAX-SAT heuristic and upon it's poor results, motivated us to search for a more robust algorithm — which ended up being the Genetic Algorithm. Therefore, it is worth mentioning that not much effort was put into to improving it, nevertheless we found it valuable to present and compare it to the other developed algorithms.

The Taboo Search we developed is essentially a local search which bans moves. In it, a solution is represented by an array of boolean values and neighbors are generated

by flipping one or two bits from a solution. In each iteration, after generating a set of $N\_NEIGHBORS$ neighbors, the algorithms chooses the best candidate c and checks if the move — that is, the bit flips at certain indexes of the solution — that generated c is on the taboo list. If it is, then the algorithm skips to the next iteration, otherwise it compares it with the current best solution, replaces it in case c is better, and bans the move. The algorithm also has an aspiration criteria, which disregards the taboo list, in case the generated solution satisfies all the clauses. A pseudo-code of Taboo Search is shown in Algorithm 1.

The maximum number of iterations is $MAX\_ITERATIONS \cdot C$ where C is the number of clauses. This is because C is the maximum number of times our solution can be improved. The most expensive part of each iteration is to select the best neighbor, which costs $O(C \cdot V)$ (where V is the number of variables), since it runs an evaluation function on a solution, which replaces each variable in each clause set with its respective value. Doing that for each solution costs $N\_NEIGHBORS \cdot VC$, so the algorithm runs at a total complexity of $O(MAX\_ITERATIONS \cdot N\_NEIGHBORS \cdot VC^2)$. Therefore, though incrementing these parameters improves the solutions generated, it also drastically increases the runtime, being worse than an exact algorithm in small test cases.

Algorithm 1 Taboo Search

```
function TS(ClauseSet)
    TabooList ← ∅
    s ← A random solution
    while MAX_ITERATIONS without improvement do
        Generate N_NEIGHBORS neighbor solutions
        b ← Select best neighbor
        if b satisfies ClauseSet then
            return (true, b)
        else
            m ← b's move
            if m is not in TabooList and b is better than s then
                s ← b
            end if
            Add m to TabooList.
        end if
    end while
    return (false, b)
end function
```

4.2. Genetic Algorithm

Our genetic algorithm is based on the work of [Bhattacharjee and Chauhan 2017]. For our version of this algorithm, each individual of the population is an array of boolean values representing a SAT solution. The fitness of a solution is the number of clauses it satisfies. The `crossover` of two individuals is a variable assignment that for each variable it gets the value from one of the parents, chosen randomly. Finally, a `mutation` just flips the value assigned for one variable. The algorithm goes as shown in Algorithm 2.

Algorithm 2 Genetic Algorithm

```
function FINDSAT
    S ← A random population of size POP_SIZE
    while MAX_ITERATIONS without improvement do
        Calculate the fitness of the population
        Sort through fitness
        Keep the ELITISM_RATE% best solutions and eliminate the rest
        Update the best solution
        while Size of population != POP_SIZE do
            a, b ← Solutions from the best, selected using Roulette Wheel method
            c ← crossover(a, b)
            c ← mutation(c)
            Add c to the next generation
        end while
    end while
end function
```

The maximum number of iterations is $MAX\_ITERATIONS \cdot C$ where C is the number of clauses. This is because C is the maximum number of times our solution can be improved. The most expensive part of each iteration is to calculate the fitness of a solution, which costs $O(VC)$ (V is the number of variables). Doing that for each solution costs $POP\_SIZE \cdot VC$. So, the algorithm runs at a total complexity of $O(MAX\_ITERATIONS \cdot POP\_SIZE \cdot VC^2)$. Therefore, similarly to Taboo Search, though incrementing these parameters improves the solutions generated, it also drastically increases the runtime, being worse than an exact algorithm in small test cases.

5. Experiment description and results

For this work, we test and compare four algorithms. Among them, one is exact (DPLL with DLIS as heuristic to select variables), one is a common heuristic (CIDPLL, presented in previous works) and the remaining ones are the meta-heuristic approaches presented in this report. All algorithms were implemented using C++17.

The tests were executed in three groups of test cases: 20 variables and 91 clauses, 50 variables and 218 clauses, 125 variables and 538 clauses. Each group contains 100 test cases and each test case was executed 5 times. All test cases are from SATLIB [Hoos and Stützle 2024]. All of the formulas tested are satisfiable, so the correct answer is the number of clauses.

For the exact algorithm DPLL, it was inviable to measure the runtime with 125 variables, because of its complexity.

For the genetic algorithm, the following parameters were set:

- POP_SIZE = 200
- ELITISM_RATE = 50%
- MAX_ITERATIONS = 1200

For the Taboo Search, the following parameters were set:

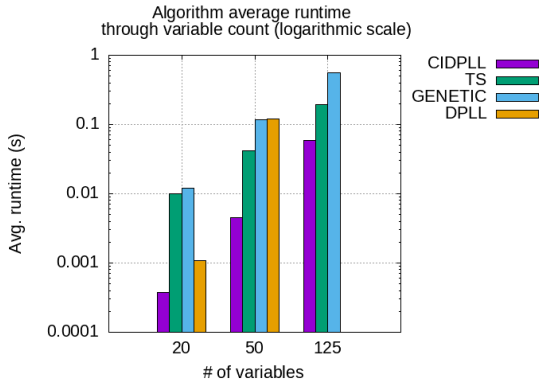
- N_NEIGHBORS = 12
- MAX_ITERATIONS = 200

The experiments where run on a machine with the following specifications:

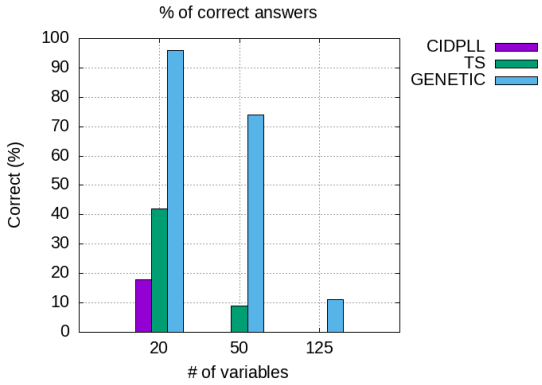
- System type: 64x
- Processor: Intel(R) Core(TM) i7-11390H CPU @3.4GHz
- Memory: 16GB
- OS: Ubuntu 22.04.5 LTS
- Language: C++ version 17 compiled with g++ version 11.4.0, utilizing -O3 flag.

For each test case, we measured the runtime of each execution. In addition, we measured the maximum number of clauses satisfied by the end of each execution. Finally, we calculate whether the solution is correct and its error margin towards the correct answer.

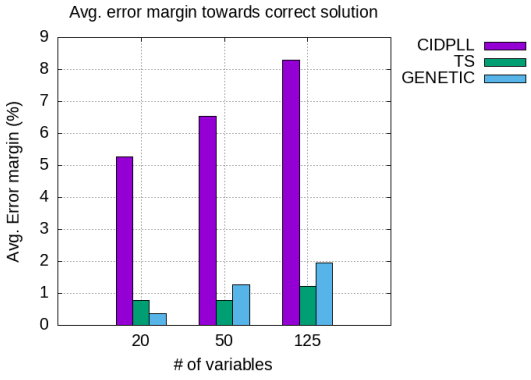
With these data we computed, for each algorithm, the average error margin from the best solution of each test case. In addition, we also measured the the average run time for each group and the percentage of correct answers. The data can be visualized at Figures 1a, 1b, 1c. The raw data is present at the appendix, in Tables 1, 3, 2.



(a) Avg. runtime (log scale)



(b) % of correct answers



(c) Avg. error margin

Figure 1: Results of all algorithms

The code and the test instances are stored in the work's GitHub repository [Gabriel and Eduardo 2024].

6. Result discussion

For small test cases, the exact algorithm had a viable runtime, making it the superior algorithm for the cases of 20 and 50 variables. Although its complexity on worst case is above $O(2^v)$, we could see that in average it does not execute that much operations. This happens because of the several optimizations like tree prunning and DLIS which makes the search go to less costly branches.

Among the meta-heuristics, we could see that for small test cases, the genetic algorithm provided most correct solutions. Consequentially, the genetic algorithm provided smaller error margins. All of that with its average runtime being worse than the taboo search by merely 0.001 seconds. Therefore, for small test cases the genetic algorithm was superior.

For larger test cases, the number of correct answers decays drastically for the meta-heuristics. However, the error margin keeps being small, with the worst average being of only 1.9%. This means that the algorithms keep obtaining high quality solutions, even if they dont get any correct one.

One interesting observation is that the taboo search got less correct answers on large test cases. However, its solutions were more stable, showing an lower average error margin. The reason for this can be easily visualized in figure 2, where the taboo search solutions got more clustered, but rarely hit the top.

Another fact is that the genetic runtime for 125 variables was very worse than the taboo search. This can indicate that the genetic algorithm could become unsuitable very fast for a large number of variables. If this is validated, it can mean that as the number of variables grows, the taboo search becomes more viable over the genetic because of the difference in runtime.

The CIDPLL was the fastest algorithm among all. However, it presented few correct answers. In addition, its answers had the most margin of error towards the correct solution. Therefore, this algorithm was left behind against the meta-heuristic approaches.

Overall, we believe both meta-heuristic algorithms performed really well in both runtime and quality of solutions. For small test cases, it is easy to see the genetic algorithm was superior, but for larger ones, we could not provide statistical tools to decide if one is better than another. However, we can see that the solutions of the taboo search are more stable, while better solutions can be achieved with the genetics. Therefore, both algorithms proved suitable for different contexts as they have strong and weak points.

7. Conclusion

This is our last work for the subject of Algorithms Project and Analysis. In this work, we researched about the state of art of attempted meta-heuristics for solving SAT/MAX-SAT problem. In addition we presented our own attempts with two meta-heuristics, a genetic algorithm and a taboo search algorithm. Finally, we executed these algorithms against SATLIB testcases and analysed the results.

Although both meta-heuristics rarely provided the correct solutions, they still provided high quality answers with a low margin of error from the optimal solution. We could see that the taboo search proved to be more stable, but the genetic algorithm got

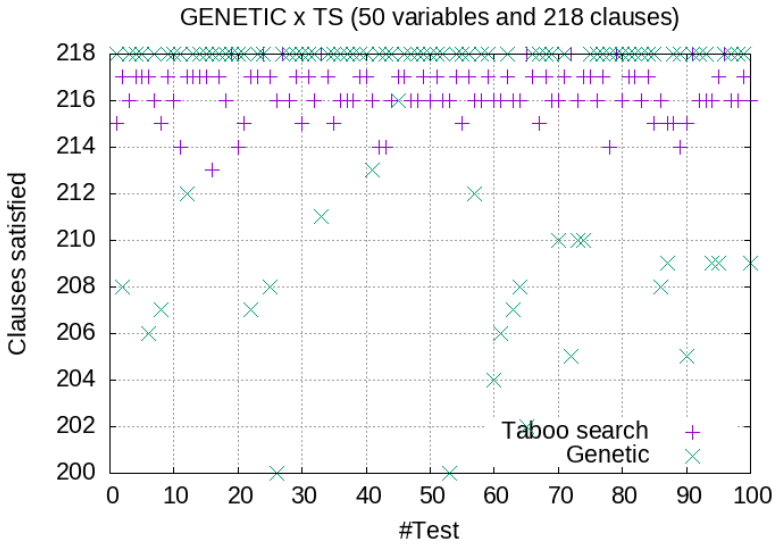


Figure 2: GENETICS x TS on 50 variables and 218 clauses

more correct answers. Therefore, both algorithms had good qualities that made them viable at least for cases with more than 100 variables, as the exact algorithm still has viable runtime up to 50 variables.

With this work, we gained a lot of practical knowledge of designing, implementing and testing meta-heuristic approaches to NP-Hard problems. Unfortunately, we could not provide inferential statistics tools to derive conclusions about the algorithms. However, we want to make more precise analysis for future projects. Finally, we hope to gather experience to design better heuristics to contribute to the state-of-art, as these heuristics did not perform better than the literature.

References

[Alkasem and Menai 2021] Alkasem, H. H. and Menai, M. E. B. (2021). A stochastic local search algorithm for the partial max-sat problem based on adaptive tuning and variable depth neighborhood search. *IEEE Access*, 9:49806–49843.

[Bhattacharjee and Chauhan 2017] Bhattacharjee, A. and Chauhan, P. (2017). Solving the SAT problem using Genetic Algorithm. *Advances in Science, Technology and Engineering Systems Journal*, 2(4):115–120.

[Boughaci and Drias 2005] Boughaci, D. and Drias, H. (2005). Efficient and experimental meta-heuristics for max-sat problems. In Nikoletseas, S. E., editor, *Experimental and Efficient Algorithms*, pages 501–512, Berlin, Heidelberg. Springer Berlin Heidelberg.

[Bouhmala 2012] Bouhmala, N. (2012). A multilevel memetic algorithm for large sat-encoded problems. *Evolutionary Computation*, 20(4):641–660.

[Bouhmala 2014] Bouhmala, N. (2014). A variable neighborhood walksat-based algorithm for max-sat problems. *The Scientific World Journal*.

[Bouhmala 2016] Bouhmala, N. (2016). A variable neighbourhood search structure based-genetic algorithm for combinatorial optimisation problems. *International Journal of Intelligent Systems Technologies and Applications*, 15(2):127–146.

[Bouhmala 2019] Bouhmala, N. (2019). Combining simulated annealing with local search heuristic for max-sat. *Journal of Heuristics*, 25(1):47–69.

[Cai 2015] Cai, S. (2015). Balance between complexity and quality: local search for minimum vertex cover in massive graphs. In *Proceedings of the 24th International Conference on Artificial Intelligence*, IJCAI'15, page 747–753. AAAI Press.

[Cai and Lei 2020] Cai, S. and Lei, Z. (2020). Old techniques in new ways: Clause weighting, unit propagation and hybridization for maximum satisfiability. *Artificial Intelligence*, 287:103354.

[Chu et al. 2023] Chu, Y., Cai, S., and Luo, C. (2023). Nuwls: Improving local search for (weighted) partial maxsat by new weighting techniques. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(4):3915–3923.

[CHU et al. 2019] CHU, Y., LUO, C., CAI, S., and YOU, H. (2019). Empirical investigation of stochastic local search for maximum satisfiability. *Frontiers of Computer Science*, 13(1):86.

[Cook 1971] Cook, S. A. (1971). The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, STOC '71, page 151–158, New York, NY, USA. Association for Computing Machinery.

[CoreO Research Group 2016] CoreO Research Group, U. o. H. (2016). Maximum satisfiability (maxsat): Aaai 2016 tutorial. `https://www.cs.helsinki.fi/group/coreo/aaai16-tutorial/`. Accessed: 2025-01-09.

[Djenouri et al. 2019] Djenouri, Y., Habbas, Z., Djenouri, D., and Fournier-Viger, P. (2019). Bee swarm optimization for solving the maxsat problem using prior knowledge. *Soft Computing*, 23(9):3095–3112.

[Gabriel and Eduardo 2024] Gabriel, I. and Eduardo, C. (2024). Paa project. `https://github.com/CinquilCinquil/PAA-Trabalho-1`.

[Hoos and Stützle 2024] Hoos, H. H. and Stützle, T. (2024). Satlib benchmark problems. `https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html`. Accessed: 2024-11-12.

[Luo et al. 2015] Luo, C., Cai, S., Wu, W., Jie, Z., and Su, K. (2015). Ccls: An efficient local search algorithm for weighted maximum satisfiability. *IEEE Transactions on Computers*, 64:1830–1843.

[Miyashiro and Matsui 2006] Miyashiro, R. and Matsui, T. (2006). Semidefinite programming based approaches to the break minimization problem. *Computers & Operations Research*, 33(7):1975–1982.

[Sandholm 1999] Sandholm, T. (1999). An algorithm for optimal winner determination in combinatorial auctions. In *IJCAI'99: Proceedings of the 16th International Joint Conference on Artificial Intelligence*, pages 542–547, San Francisco, CA, USA. Morgan Kaufmann Publishers Inc.

[Strickland et al. 2005] Strickland, D. M., Barnes, E., and Sokol, J. S. (2005). Optimal protein structure alignment using maximum cliques. *Operations Research*, 53(3):389–402.

[Whitley et al. 2013] Whitley, D., Howe, A., and Hains, D. (2013). Greedy or not? best improving versus first improving stochastic local search for maxsat. *Proceedings of the AAAI Conference on Artificial Intelligence*, 27(1):940–946.

[Xu et al. 2002] Xu, H., Rutenbar, R. A., and Sakallah, K. (2002). Sub-sat: A formulation for relaxed boolean satisfiability with applications in routing. In *ISPD '02: Proceedings of the 2002 International Symposium on Physical Design*, pages 182–187, New York, NY, USA. ACM.

8. Appendix

Variables	Clauses	Avg. Runtime			
		DPLL	CIDPLL	GENETIC	TS
20	91	0.001	0.0003	0.011	0.010
50	218	0.118	0.004	0.118	0.042
125	538	-	0.058	0.559	0.191

Table 1: Avg. Runtime per algorithm

Variables	Clauses	Avg. Error margin (%)		
		CIDPLL	GENETIC	TS
20	91	5,2	0,3	0,7
50	218	6,5	1,2	0,7
125	538	8,3	1,9	1,2

Table 2: Avg. Error margin per algorithm

Variables	Clauses	Correct answers (%)		
		CIDPLL	GENETIC	TS
20	91	18	96	42
50	218	0	74	9
125	538	0	11	0

Table 3: Correct answers % per algorithm

All detailed results are located in: `https://drive.google.com/drive/folders/1kO_NRPv9HAH5AxHYISPmh5Da-pP3K5km?usp=sharing`